FOCUS System Design Plan

Phase 5 Critical Task and Alert Engine | Phase 7 Focus Guard

Offline-first architecture for a Flutter and Dart personal productivity OS on Android and Windows

Design position. Keep task truth and policy in a shared Dart core; delegate OS scheduling and focus interruption to small native adapters. Reminders can be persistent and adaptive, but neither platform lets an ordinary app guarantee forced wake, silence override, or unescapable control of other apps. Design for transparent permissions, durable local state, and explicit fallbacks when the OS declines delivery.

1. Purpose and boundaries

Phase 5 turns a user-created critical task into a durable local alert schedule, tracks delivery and acknowledgement, and escalates only within user-selected limits. Phase 7 observes a narrowly configured set of distracting apps or websites during a chosen focus session, then presents a brief “You are distracted. Return to your task.” intervention and a deliberate return path.

The system remains useful without an account, network, or cloud service. Local database state is authoritative; OS notification queues and alarms are projections rebuilt from that state. Phase 5 provides reminders, not coercive device control. Phase 7 provides a user-approved prompt, not a kiosk lock.

2. Product decisions and non-goals

Decision	Design rule
Critical means user-designated	A task is critical only when the user marks it so and chooses reminder behavior.
Do not defeat system controls	Never change ringer, DND, volume, lock state, permissions, or accessibility settings on the user’s behalf.
Focus intervention is interruptible	User can dismiss, snooze, pause, end the session, or disable monitoring at any time.
No screen recording by default	Detect app/window identity from OS events. Never capture pixels, keystrokes, message content, or full accessibility trees.
No cloud dependency	Tasks, rules, events, and sessions are local. Sync is outside these phases.
No app-level guarantee	Power-off, force-stop, revoked access, OEM battery rules, disabled notifications, or DND can suppress or delay behavior.

3. System context and architecture

Use a layered Flutter application with a Dart domain core and native platform adapters. SQLite is the durable source of truth. A schedule reconciler computes desired OS schedules from active rules and compares them with adapter state. Adapters report capability and permission state; they do not decide policy.

Flutter UI: task editor, alert settings, permission education, focus controls, local event history

Dart application layer: TaskCommandService | AlertPolicyEngine | ScheduleReconciler | FocusSessionCoordinator

Dart domain core: Task, AlertRule, AlertOccurrence, FocusSession, FocusPolicy, FocusEvent, PermissionSnapshot

Local persistence: SQLite (Drift recommended), migrations, transactional outbox, append-only event history

Platform boundary: versioned MethodChannel/Pigeon contracts; idempotent commands with stable IDs

Android: Kotlin AlarmManager/NotificationManager + optional AccessibilityService + optional overlay

Windows: Windows App SDK notifications/scheduler + optional foreground watcher + owned prompt window

Keep the common core free of platform APIs. Normalize platform callbacks into typed domain events and persist them before notifying the UI. Closing Flutter must not lose an occurrence or active session.

4. Phase 5 Critical Task and Alert Engine

4.1 Responsibilities

- Create a critical task with due time, recurrence, timezone policy, acknowledgement rule, and bounded escalation plan.
- Materialize upcoming occurrences and schedule them with an OS adapter; reconcile after edits, reboot, time change, permission change, or app start.
- Deliver privacy-safe notifications with Open, Done, Snooze, and optional “I am getting ready” actions.
- Record scheduled, fired, presented, actioned, missed, suppressed, and failed states. OS acceptance is not proof the user saw or heard an alert.
- Offer voice reminders only when opted in and permitted. Respect actual route and volume; never raise volume or bypass DND/silent mode.

4.2 Core data model

Entity	Important fields	Invariant / purpose
Task	id UUID, title, notes?, criticality, status, createdAt, updatedAt, deletedAt?	Local canonical work item; notes stay out of lock-screen payload by default.
TaskSchedule	taskId, localDateTime, zoneId, recurrenceRule?, duePolicy, graceMinutes	Wall-clock intent and timezone stay separate from resolved UTC instant.
AlertRule	id, taskId, leadMinutes, channel, escalationSteps[], quietHoursPolicy, enabled	User-selected behavior; escalation count and intervals have caps.
TaskOccurrence	id, taskId, plannedAtUtc, localDate, zoneId, revision, state, acknowledgedAt?	Stable instance; unique task/date/recurrence slot.
AlertAttempt	id, occurrenceId, ordinal, plannedAtUtc, attemptedAtUtc?, channel, result, reasonCode?	Audit one attempt. Presented does not prove perception.
Snooze	occurrenceId, untilUtc, count, source	At most one active snooze per occurrence; bounded.
PlatformSchedule	scheduleKey, occurrenceId, revision, platformToken?, desiredAtUtc, syncedAtUtc?, status	Rebuildable projection; not source of truth.
PermissionSnapshot	capability, granted, checkedAt, platformVersion, detailCode?	Cached for UI only; re-check before sensitive scheduling.

Use UTC epoch milliseconds for resolved instants and retain IANA zone ID plus local wall time. Recurring 5:00 a.m. tasks follow local wall time by default. For a skipped or repeated daylight-saving time, resolve to the next valid local instant and record the policy. Timezone changes recalculate future occurrences; completed occurrences remain immutable.

4.3 Alert state machine

State	Meaning	Next states
Draft	Task/rule inactive	Scheduled, Cancelled
Scheduled	Occurrence + OS projection committed	Firing, Cancelled, NeedsReconcile
Firing	Adapter attempting delivery	Presented, Suppressed, Failed, NeedsReconcile
Presented	OS accepted/surfaced; perception unknown	Acknowledged, Snoozed, Escalating, Expired
Snoozed	User selected bounded delay	Scheduled, Acknowledged, Expired
Escalating	Next opted-in step due	Presented, Suppressed, Failed, Acknowledged, Expired
NeedsReconcile	OS state uncertain after interruption	Scheduled, Cancelled, Failed
Acknowledged	Done/ready/explicit acknowledgement	Terminal
Expired	Grace window ended; history retained	Terminal

State	Meaning	Next states
Cancelled	Task/rule disabled/deleted	Terminal

Transitions use occurrence revision and current state. Every schedule command is idempotent on (occurrenceId, revision, step); callbacks from older revisions are ignored. Recurrence creates a new occurrence, never reopens a terminal one.

4.4 Delivery and escalation policy

– At save, commit Task, schedule, rule, next occurrence, and outbox entry in one database transaction.

– Reconciler computes the desired alarm set, upserts deterministic keys, and cancels obsolete revisions.

– On fire, native receiver validates key/revision, persists delivery event, then requests notification presentation.

– Actions carry opaque occurrence IDs and revision; validate current state before applying.

– If unacknowledged after grace, compute next allowed step. Stop at the user-selected maximum. Never loop indefinitely.

– On restart, rebuild from SQLite. Do not replay all missed alerts; deliver one concise overdue summary or mark stale instances missed.

Recommended default escalation: one due-time notification and one follow-up after a selected interval, then stop. Voice and alarm-like presentation are opt-in and subject to OS capability and policy. Quiet-hour choices may defer, show silently, or show a normal notification; never promise DND override.

4.5 Android adapter

Capability	Implementation	Permission / limitation
Normal reminder	AlarmManager inexact alarm when timing is flexible	Exact access only when genuinely time-critical. Android 12+ requires applicable access; check before use.
Visible alert	NotificationManager channel with actions	POST_NOTIFICATIONS runtime grant on Android 13+; can be denied, silenced, grouped, or hidden.
Alarm-like surface	Alarm-clock semantics only for genuine alarm-clock behavior	Android 14+ full-screen intent access is restricted to eligible alarm/calling apps; do not rely on lock-screen launch for productivity.
Reboot/time change	BOOT_COMPLETED and time/timezone receivers trigger reconciliation	Do not assume alarms survive reboot; keep receivers small.
Voice/audio	Notification sound or user-started TTS	Cannot raise volume or promise sound through silent mode, DND, Bluetooth, or disabled channel.

Keep notification channel IDs stable. Use AlarmManager for upcoming occurrences; use WorkManager for deferred maintenance, not minute-accurate alarms. Request exact-alarm access only in context, after explaining value and confirming distribution policy eligibility.

4.6 Windows adapter

• For new desktop work, prefer Windows App SDK AppNotificationManager. Scheduled notifications fit known future reminders, but may be dropped if the PC stays off beyond the delivery window.

• For restart recovery, persist occurrences and reconcile after resume/logon. Use an MSIX background task or narrowly scoped Task Scheduler integration only when lifecycle requirements justify it.

• Windows Focus / Do Not Disturb can silence notifications. A normal desktop app cannot force audio, change focus settings, or guarantee foreground activation.

• Scheduled delivery and background execution have different guarantees; record adapter outcomes and expose limitations in the UI.

5. Phase 7 Focus Guard

5.1 Responsibilities and detection strategy

Focus Guard runs only during a session started or scheduled by the user. The user selects target task, duration, monitored apps or domains where supported, intervention delay, prompt style, and escape controls. Detection uses app/window identity, not inferred intent. “Distracting” is a user-defined package/process allowlist; do not classify pixels, topics, or behavior.

5.2 Focus data model

Entity	Important fields	Purpose
FocusPolicy	id, name, monitoredTargets[], graceSeconds, cooldownSeconds, blockMode, promptText	User-owned settings; v1 blockMode is reminder or overlay prompt.
FocusSession	id, policyId, taskId, startedAtUtc, plannedEndUtc, endedAtUtc?, state, source	One active session per profile; resume behavior is explicit.
FocusEvent	id, sessionId, occurredAtUtc, kind, targetAppId?, decision, triggerId	Minimal history; no window title or document text by default.
Cooldown	sessionId, targetAppId, untilUtc, reason	Prevents repeated prompts on app-switch events.
AdapterHealth	platform, serviceState, lastHeartbeat?, lastErrorCode?	Detects disabled service, access revoke, crash, unsupported path.

5.3 Focus Guard state machine

State	Entry condition	Behavior / transition
Inactive	No session	No monitoring; start → capability request or monitoring.
RequestingCapability	User enables automatic feature	Explain need, open settings; cancel remains safe.
Monitoring	Session active and adapter ready	Process target-app events; target → Candidate.
Candidate	Target observed	Debounce, validate active session/foreground/cooldown; then Prompting.
Prompting	Intervention qualified	Show prompt. Dismiss/snooze → Cooldown; end → Inactive.
Cooldown	Prompt acknowledged/dismissed	Minimal monitoring; timer expiry → Monitoring.
Paused	User paused or exempt condition	No intervention; resume explicitly or by policy.
Degraded	Permission revoked/service stopped/unsupported	Truthful unavailable status; manual mode remains.
Completed	Timer ends or user ends	Stop adapter, record summary, release overlay/window.

Qualification is: active session AND healthy adapter AND foreground target in user allowlist AND dwell above debounce AND no cooldown/exemption. Recheck foreground immediately before prompt display. A stale event must never cover an unrelated app.

5.4 Android adapter and permission experience

– Explain that automatic mode observes foreground app identity and event timing for selected packages. State that screen contents are not collected.

– Offer a lower-permission alternative first: app-local focus timer with manual check-in, or notification reminder.

– If automatic detection is enabled, direct the user to enable AccessibilityService. Configure minimum event types and package filters; avoid window content retrieval and do not log node text. Accessibility APIs are sensitive, intended for assistive purposes, and require distribution-policy review.

– Request display-over-other-apps separately and only for optional overlay mode. Prefer notification or app-local return flow by default.

– On service disconnect, process death, package update, or revoke, enter Degraded, remove overlay, and show truthful status next time. Never silently re-enable access.

Do not use AccessibilityService to block taps, simulate repeated gestures, close apps, or trap the user. The dismissible prompt can offer Return to task, Snooze 5 min, Pause, and End session. Keep system navigation and emergency access available.

5.5 Windows adapter

- A native watcher may sample foreground process identity during an active session; prefer event-driven WinEvent hooks where stable. Do not store window titles or content.
- For v1, report monitoring active only while a visible tray companion is running. Do not promise unattended monitoring before validating logoff, sleep, crash, update, and reboot lifecycle.
- Show the prompt in an owned topmost window or notification. Foreground rules may prevent activation; queue a notification or record a missed intervention instead.
- Do not install shell hooks, keyboard filters, global input blocks, or system policies. Kiosk/Assigned Access is managed-device scope and outside this consumer app.
- Allow exemptions for password managers, calls, accessibility tools, system settings, and user-selected apps. The user can pause or quit the watcher.

6. Flutter and native contracts

Use generated Pigeon APIs where practical; otherwise define versioned MethodChannel messages. Commands return typed outcomes. Each operation includes requestId and idempotencyKey; adapter events include platform and adapterVersion.

Contract	Request / result summary
scheduleOccurrence	occurrenceId, revision, fireAtUtc, policy → Scheduled | PermissionRequired | Unsupported | Failed(code)
cancelOccurrence	occurrenceId, revision → Cancelled | AlreadyAbsent | Failed(code)
getCapabilities	→ exactScheduling, notifications, accessibility, overlay, backgroundWatcher, checkedAt
startFocusMonitoring	sessionId, policySnapshot → Started | PermissionRequired | Unsupported | Failed(code)
stopFocusMonitoring	sessionId → Stopped | AlreadyStopped
focus event stream	ForegroundTargetChanged, ServiceStateChanged, PromptAction, AdapterError; monotonic sequence

Dart owns policy and validates requests against SQLite. Native code owns OS handles, callbacks, and immediate presentation. On restart, Dart reconciles orphaned schedules. Unknown contract versions fail closed to reminder-only/manual mode.

7. Permissions and onboarding matrix

Capability	Needed for	Request timing	Degraded behavior
Android notifications	Visible reminders/actions	After first alert is created	Due list and capability warning; no precise delivery claim.
Android exact alarm access	Precise alarm during idle	After user selects precision and gets explanation	Use inexact alternative and disclose approximate timing.
Android AccessibilityService	Automatic foreground detection	Only when enabling automatic guard	Manual timer; automatic mode unavailable.
Android overlay access	Prompt outside app	Only after user chooses overlay	Notification/app-local screen fallback.
Windows notifications	OS delivered reminders	First notification use	In-app center; warn that OS can silence.
Windows watcher lifecycle	Monitoring outside UI	Opt-in with visible status	Session degrades when watcher exits.

Do not bundle capabilities into broad consent. Show what data each exposes, settings link, re-check action, and disable route. Explain that Android accessibility access is sensitive even though this implementation filters to app identity and avoids reading content.

8. Failure handling and recovery

Failure	Detection	Recovery
Notification denied/channel blocked	Capability query or adapter result	Persist task; show due list; mark delivery unavailable; settings link.
Exact access revoked	Permission broadcast/preflight	Switch to allowed inexact path; report precision change.
Power-off/force-stop	No fire callback; startup reconcile	Summarize one recent overdue occurrence; never flood backlog.
DST/timezone change	System event or next launch	Recompute future occurrences transactionally; increment revision.
Duplicate alarm callback	Unique attempt key/state guard	Ignore duplicate; retain one event.
Crash between database and OS call	Outbox remains pending	Replay idempotently next launch/callback.
Accessibility disabled/watcher crash	Disconnect or heartbeat timeout	Remove overlay; Degraded state; manual timer available.
Stale focus event	Sequence + foreground recheck	Drop; never overlay another app.
Overlay denied/foreground blocked	Presentation result/timeout	Notification fallback or missed intervention record.
Clock moved backwards	UTC instant and unique local slot	Apply documented policy; prevent duplicate fire.
Database migration fails	Schema/integrity check	Preserve backup and open recovery mode; never silently delete.

9. Security, privacy, and retention

- Store task and focus records in app-private storage. Encrypt at rest with platform-protected keys if threat model requires it; exclude keys from backup and document local export/recovery.
- FocusEvent stores only timestamp, target package/process ID, session ID, and decision. Default detail retention: 30 days, configurable and deletable. Never store full titles, accessibility trees, screenshots, URLs, or keystrokes.
- Notifications use generic lock-screen text unless task names are explicitly allowed. Keep notes out of intents. Action payloads use opaque IDs and are validated against current revision/state.
- Use explicit immutable Android PendingIntents where applicable; avoid exported receivers unless required and validate inputs.
- On Windows, minimize process data, do not persist window handles, secure local IPC, and limit watcher commands to authenticated app processes.
- Keep logs local and redacted. Provide delete-all and export controls. No telemetry by default; future diagnostics must be opt-in and content-free.

10. Test plan

10.1 Unit and property tests

- Alert recurrence, time zones, DST gaps/overlaps, grace, quiet hours, bounded escalation, snooze cap, and terminal transitions.
- Idempotency under duplicate callbacks, repeated reconcile, stale revisions, outbox replay, and late user actions.
- Focus debounce/cooldown, allowlist/exemptions, session boundaries, event ordering, and stale foreground events.
- Properties: terminal occurrence never reopens; attempts stay within limit; no prompt after stop; one active schedule per key.

10.2 Adapter and integration tests

- Android: notifications allowed/denied; exact access allowed/revoked; reboot; timezone changes; Doze; battery saver; delayed callbacks; accessibility connect/disconnect; overlay allow/deny; TalkBack coexistence.
- Windows: scheduled notification delivery window; notifications disabled/Focus active; sleep/resume; logoff/reboot; watcher crash/restart; foreground races; elevation; DPI/multi-monitor; topmost failure.
- Cross-platform contract versioning, malformed payloads, sequence duplicates, migration recovery, and permission downgrade.
- Keyboard-only and screen-reader operation, large text, prompt announcements, no focus trap, localization, and easy pause/disable.

10.3 Acceptance scenarios

Scenario	Expected result
College task at 5:00 a.m.; phone silent	Task stays scheduled; notification may be silent. No volume change or claim of audible delivery; overdue state appears on next open.
User denies exact alarms	Task saves; use inexact behavior when suitable and label timing approximate.
Target short-video app opens in session	After debounce one prompt appears; return/snooze/pause/end work; no video content is inspected.
User disables Accessibility mid-session	Guard degrades, removes overlay, and reports monitoring is off.
Windows notification silenced by Focus	Occurrence recorded/reconciled; no attempt to change Focus settings.
App crashes after save	Outbox replay reconstructs schedule exactly once.

11. Ordered implementation milestones

– Define target Android SDK and Windows packaging; settle recurrence/DST semantics, defaults, escalation caps, focus targets, permission rationale.

– Implement Dart entities, validators, state machines, clock/timezone abstraction, and deterministic policy tests.

– Build Drift schema/migrations, transactional handlers, outbox, repositories, event history, and recovery behavior.

– Build Phase 5 UI: task editor, alert profile, notification privacy, permission status, due list, acknowledge/snooze, history.

– Implement Android alert adapter: notification channels/actions, inexact scheduling, optional exact gate, boot/time reconciliation, capability probes.

– Implement Windows alert adapter: AppNotificationManager, scheduled notifications or packaged background mechanism, restart reconciliation, activation.

– Run Phase 5 failure-injection and acceptance checks; expose delivery capability rather than guaranteed-alarm labels.

– Implement Phase 7 manual mode first: session/policy editor, task link, timer, prompt UI, pause/end, summaries, state machine.

– Add Android foreground package observation with minimum AccessibilityService config, settings handoff, health, debounce, and fallback; add optional overlay only after policy/usability review.

– Add Windows watcher behind explicit opt-in with visible tray status; process identity only; prompt and notification fallback.

– Complete privacy/security review, adapter conformance, accessibility QA, battery tests, revoke/reboot tests, and opt-in rollout diagnostics.

12. Release gates and operational measures

- Phase 5 demonstrates no duplicate logical attempts under callback replay and recovers after reboot/timezone change.
- Each platform distinguishes scheduled, OS-accepted, suppressed, failed, and unknown delivery; never claim the user heard or woke.
- Focus observation scope is documented in onboarding; screen content is neither captured nor uploaded.
- Every capability can be revoked without crash, persistent overlay, or misleading active status.

- Local health measures: reconcile outcomes, notification actions, permission state, focus prompt counts, and adapter health. Keep on-device unless privacy-reviewed export is explicitly enabled.

13. Open implementation choices

Choice	Recommended default	Decide before
Flutter database	Drift over SQLite	Schema implementation
Android exact-alarm grant	SCHEDULE_EXACT_ALARM only when user-selected precision genuinely requires it; evaluate USE_EXACT_ALARM only after eligibility review	Distribution and alarm semantics
Android Focus observation	AccessibilityService only after policy review; otherwise manual mode	Store submission
Windows background model	MSIX with Windows App SDK or explicitly scoped Task Scheduler companion	Deployment architecture
Focus prompt	Dismissible reminder/overlay with obvious escape	Phase 7 UI
Retention	30-day event detail; aggregate history until deleted	Privacy review

14. Platform references

- Android schedule alarms: https://developer.android.com/develop/background-work/services/alarms
- Android exact alarm changes: https://developer.android.com/about/versions/14/changes/schedule-exact-alarms
- Android notification runtime permission: https://developer.android.com/develop/ui/compose/notifications/notification-permission
- Android accessibility service guidance: https://developer.android.com/guide/topics/ui/accessibility/service
- Windows notifications overview: https://learn.microsoft.com/en-us/windows/apps/develop/notifications/
- Windows scheduled app notifications: https://learn.microsoft.com/en-us/windows/apps/develop/notifications/app-notifications/app-notifications-scheduled
- Windows App SDK background tasks: https://learn.microsoft.com/en-us/windows/apps/windows-app-sdk/applifecycle/background-tasks
- Windows Focus session state: https://learn.microsoft.com/en-us/windows/apps/windows-app-sdk/applifecycle/focus-session